

Can We Enhance Bug Report Quality Using LLMs?: An Empirical Study of LLM-Based Bug Report Generation

Jagrit Acharya

Electrical and Software Engineering
University Of Calgary
Calgary, Alberta, Canada
jagrit.acharya1@ucalgary.ca

Gouri Ginde

Electrical and Software Engineering
University Of Calgary
Calgary, Alberta, Canada
gouri.deshpande@ucalgary.ca

Abstract

Bug reports contain the information developers need to triage and fix software bugs. However, unclear, incomplete, or ambiguous information may lead to delays and excessive manual effort spent on bug triage and resolution. In this paper, we explore whether Instruction fine-tuned Large Language Models (LLMs) can automatically transform casual, unstructured bug reports into high-quality, structured bug reports adhering to a standard template. We evaluate three open-source instruction-tuned LLMs (*Qwen 2.5*, *Mistral*, and *Llama 3.2*) against ChatGPT-4o, measuring performance on metrics such as CTQRS, ROUGE, METEOR, and SBERT. Our experiments show that fine-tuned Qwen 2.5 achieves a CTQRS score of (77%), outperforming both fine-tuned Mistral (71%), Llama 3.2 (63%) and ChatGPT in 3-shot learning (75%). Further analysis reveals that Llama 3.2 shows higher accuracy of detecting missing fields particularly Expected Behavior and Actual Behavior, while Qwen 2.5 demonstrates superior performance in capturing Steps-to-Reproduce, with an F1 score of 76%. Additional testing of the models on other popular projects (e.g., Eclipse, GCC) demonstrates that our approach generalizes well, achieving up to 70% CTQRS in unseen projects' bug reports. These findings highlight the potential of instruction fine-tuning in automating structured bug report generation, reducing manual effort for developers and streamlining the software maintenance process.

CCS Concepts

• **Software and its engineering** → *Software maintenance tools*.

Keywords

Bug report quality, large language models, instruction fine-tuning, software maintainance, and software engineering

ACM Reference Format:

Jagrit Acharya and Gouri Ginde. 2025. Can We Enhance Bug Report Quality Using LLMs?: An Empirical Study of LLM-Based Bug Report Generation. In *Evaluation and Assessment in Software Engineering (EASE '25)*, June 17–20, 2025, Istanbul, Turkiye. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3756681.3756995>



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

EASE '25, Istanbul, Turkiye

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1385-9/25/06

<https://doi.org/10.1145/3756681.3756995>

1 Introduction

Bug reports are essential in software maintenance, providing developers with critical information to identify, triage, and resolve software defects [77]. A bug report is a record of a software fault or defect that is created by an end-user or a tester [2]. However, the effectiveness of bug reports is often hindered by ambiguity, incompleteness, or inconsistency in the information provided by reporters [3, 37]. Well-structured reports that clearly articulate observed behavior (OB), expected behavior (EB), and steps to reproduce (S2Rs) minimize ambiguity and enable developers to resolve issues without much discussion & clarification [68].

Challenges in bug reporting persist due to factors such as varying reporter experience and difficulty in providing essential details like reproduction steps and expected outcomes [15, 33, 48, 67]. Adding to this problem, the lack of tool support during report creation further undermines accuracy [19, 34]. To address these challenges researchers have explored improving bug report quality by detecting weak descriptions [59]. Some studies check if a report includes key details like observed behavior, expected behavior, and steps to reproduce the issue [13, 14] using Natural Language Processing (NLP) based approaches. Others provide an overall quality assessment of the bug report and offer general suggestions for improvement [59, 77]. Advancements in NLP have led to the development of large language models (LLMs), which are transformer-based neural networks capable of predicting the next token based on the preceding context [58]. These models comprehend context and execute assigned tasks through prompts. A similar architecture has been employed by Bo et al. [6] to generate missing information in bug reports using ChatGPT. However, ChatGPT has been found to generate incorrect information [47, 63] and faces limitations due to data privacy concerns [9] in Software Engineering.

To address these limitations, we propose an approach to transforming unstructured bug reports into structured bug reports according to standardized template formats while also highlighting missing pieces of information to the reporter before submission of the bug report using open source large language models locally.

Our research contributions are as follows:

- We provide empirical evidence to show that the instruction fine-tuned LLMs perform close to state-of-the-art (SOTA) model, ChatGPT-4o (hereafter referred to as ChatGPT) in generating high-quality bug reports based on measures such as SBERT, ROUGE-1, and CTQRS (Crowdsourced Test Report Quality Score) scores.

- To the best of our knowledge, we are the first to demonstrate how LLM models can transform the reporter (natural-language-based) summary into a structured bug report as per the bug template format. This contribution is particularly significant as bug report quality directly impacts all other research domains, such as bug triage, assignments, duplication detection and prioritization.
- We show evidence for the effectiveness of Cross Platform learning (LLMs trained on bug reports from larger projects are used to generate bug reports for smaller projects). As such, our results show that instruction fine-tuned models can generalize well and perform significantly better for projects without training data.
- As a contribution to open science, we make our complete dataset and source code public for researchers to replicate our study and utilize the dataset for other explorations¹.

2 Need for this Study

This section presents a motivating example to illustrate our approach and compares it with existing methods for improving bug report quality. We investigate how instruction fine-tuned LLMs perform against state-of-the-art models like ChatGPT in generating structured bug reports, their effectiveness in generalizing across different software projects, and their ability to identify missing information while mapping summaries to structured components. Our evaluation, based on instruction fine-tuning, uses both qualitative and quantitative metrics, as shown in Table 1.

Motivating example: Figure 1 presents a sample bug report from Bugzilla, which lacks explicitly stated steps to reproduce the issue. As a result, the developer had to request clarification, leading to a delay in resolving the bug, which was ultimately fixed only after the reporter provided clear reproduction steps two months later. In contrast, Figure 2 illustrates a well-structured bug report, similar report was automatically triaged and resolved within five days, requiring minimal discussion or clarification with the reporter.

Table 1: Comparison of prior work with our approach in terms of model type, evaluation metrics employed, and adaptation techniques applied.

Papers	Model	Metrics	Adaptation
GIRT[52]	Open Source	Quantitative	Fine-tuning
ChatBR [6]	ChatGPT	Quantitative	Few-Shot
BugBlitz [72]	Open Source	Quantitative	Fine-tuning
Our Study	Open Source	Qualitative & Quantitative	Fine-tuning

3 Preliminaries

In this section, we describe the instruction fine-tuning of LLMs, the specific LLM used in our study, and the evaluation metrics CTQRS and related concepts.

Instruction fine-tuning: Instruction fine-tuning trains a language model to follow specific instructions by learning from examples [17]. For instance, we provide the model with pairs of bug

¹https://github.com/GindeLab/Ease_2025_AI_model

Sample unstructured (lacking standard format) bug report

Bug Id: 1805934

User Agent: Mozilla/5.0 (X11; Linux x86_64; rv:108.0) Gecko/20100101 Firefox/108.0

Steps to reproduce:

For a few months now, I've been suffering an intermittent problem: every now and again, all drop-down controls in Firefox would break. Menus would no longer work, drop-down selects on web pages would fail, extension menus would fail, and the hamburger menu would fail. The visible behavior is that the drop-down is drawn but then immediately erased as if I had clicked elsewhere in the window. The only fix for the problem is to restart Firefox. Recently, I realized something: every time I restarted to fix the issue, Firefox would bring up the dialog saying it was installing the latest update. And I never get a dialog to tell me that an update is available.

So what seems to be happening is: every time Firefox detects an available update, something breaks and all menus and drop-downs stop working.

Today was even worse: restarting didn't show the updating dialog, and as soon as I went to any web page, all the drop-downs broke again. So I wondered if I was wrong about the cause... but I cleared all local data (cache, cookies, the lot) and restarted one more time – and suddenly I got the updating dialog, and now drop-downs work again. Obviously this is absolutely infuriating. I'd like to do anything I can to help you track down and fix the problem.

Figure 1: This is an example of a low-quality bug report, as it does not follow the defined Bugzilla bug report template.

report summaries and well-written bug reports. By learning from these examples, the model can automatically convert any new unstructured report from the reporter into a complete, well-structured bug report following a bug report template. This differs from generic fine-tuning, which simply adapts the model to specific data without explicitly teaching it to execute instructions; instruction fine-tuning makes the model better at understanding and performing the desired transformation as per the given guidelines [40]. After fine-tuning with annotated datasets containing instructional data, language models show an enhanced ability to follow general language instructions [30]. This method, known as instruction-tuning, enhances the controllability of LLMs via natural language commands, thus significantly improving their performance and ability to generalize across unseen tasks [69, 70].

LLM Models: We conducted supervised fine-tuning using Low-Rank Adaptation (LoRA) [31] on three top-performing open-source instruction-tuned language model: unsloth/Mistral-7B-Instruct-v0.3 [22], unsloth/Qwen2.5-7B-Instruct [64], and unsloth/Llama-3.2-3B-Instruct [21]. These models were selected based on their strong ranking on the Hugging Face Open LLM Leaderboard [5] as of February 2025 and widespread use in recent studies [7, 28, 32]. They

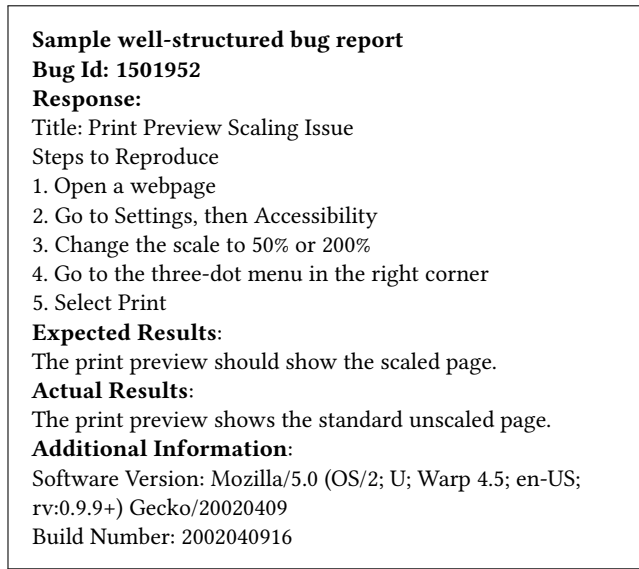


Figure 2: This is an example of a bug report generated from a fine-tuned Mistral 7B model, based on the unstructured report

also vary in size (7B and 3B parameters), allowing us to study how scale affects performance. We used the Unsloth training framework [66] for efficient fine-tuning, as it significantly reduces VRAM usage and speeds up training. For comparison, we also used Llama 3 [18] for generating unstructured bug reports and ChatGPT [53] for few-shot learning tasks.

CTQRS: CTQRS (Crowdsourced Test Report Quality Score), developed by Zhang et al. [75], is a bug-report quality assessment framework that systematically scores bug reports by combining morphological, relational, and analytical indicators through dependency parsing. We re-implemented all the 13 rules defined by the authors to determine the score of the bug reports using python.

4 Research Questions

We aim to evaluate the efficacy of instruction-fine-tuned large language models (LLMs) in generating structured, high-quality bug reports compared to general-purpose models like ChatGPT, assess their cross-project generalizability when applied to diverse open source software projects, and analyze their ability to identify missing information from user-provided bug reports while accurately mapping details to standardized bug report components.

- **RQ1: How do fine-tuned LLM models perform compared to the GPT model in generating bug reports?**

Rationale: ChatGPT models which are trained on a large corpus need little to no prior knowledge of the task in hand. Thus, through this RQ, we evaluate the effectiveness of instruction fine-tuned models against a widely used GPT model in SE research. This comparison helps determine if specialized fine-tuning provides any significant advantages over general-purpose models in generating structured, high-quality bug reports.

- **RQ2: How effective is cross-project prediction in generating structured bug reports across different software**

projects?

Rationale: Cross-project prediction enables models trained on bug reports from one project to be applied to different projects, assessing their generalizability. Evaluating a fine-tuned model trained on larger projects will help determine if learned patterns can effectively transfer across different software projects.

- **RQ3: How effective is the fine-tuned LLM in identifying missing information from summaries and mapping unstructured report information to structured bug report components?**

Rationale: Evaluating the fine-tuned LLM's effectiveness in identifying missing information mapping accuracy ensures the generated reports capture all details from the unstructured report, correctly identifies the missing information and maps them onto the respective component of bug report. It helps identify the bug report components where the LLM demonstrates the strongest and weakest effectiveness in mapping and detecting missing information.

5 Methodology

In this section, we discuss the dataset, preprocessing steps, data generation, prompt design, model fine-tuning, implementation details, and evaluation metrics used in our study (as shown in Figure 3).

Dataset and Pre-Processing: We mined a dataset comprising the recent 15,000 fixed bug reports from Bugzilla, an online bug tracking system, that were "fixed" and "closed", (as considered in the previous work [61]). The dataset was gathered using Bugzilla API over multiple iterations. First, relevant bug reports' metadata was gathered utilizing the "Get All Data" API call. Then, using the "Get All Comments" APIs, all the details regarding bug reports were gathered (Step ①). The dataset includes fields such as Bug ID, Comment ID, Comment, Priority, Severity, Status etc. Our primary focus was the Comment field, as it contained the key bug report details required for fine-tuning.

Not all bug reports contained the necessary information outlined in the Bugzilla bug report guidelines [49], which advises reporters to include steps to reproduce (S2Rs), actual results (AR), expected results (ER), and any additional relevant information in their bug report. Thus, to curate the high-quality training dataset, firstly, we employed regular expressions to filter the bugs whose bug reports had descriptions, S2Rs, EB, AB, and additional information, as shown in (Step ②). Secondly, bug reports containing stack traces or code snippets were similarly excluded. This decision was motivated by the potential for these elements to introduce noise and complexity [75], thereby negatively impacting the fine-tuning process of our model. Afterwards filtered bug reports with a CTQRS score greater than 14 as they are considered good by Zheng et al. [75] (Step ③). After these filtrations, we had 3,966 Bugs with all the required information (Step ④), out of which 200 reports were manually reviewed to check if they were of the desired quality.

Synthetic (pseudo-ground truth) data generation: Recent studies have successfully utilized LLMs to generate factually consistent summaries [62] and generate data in the domain of healthcare [24]. Taking inspiration from these studies, we first conducted multiple

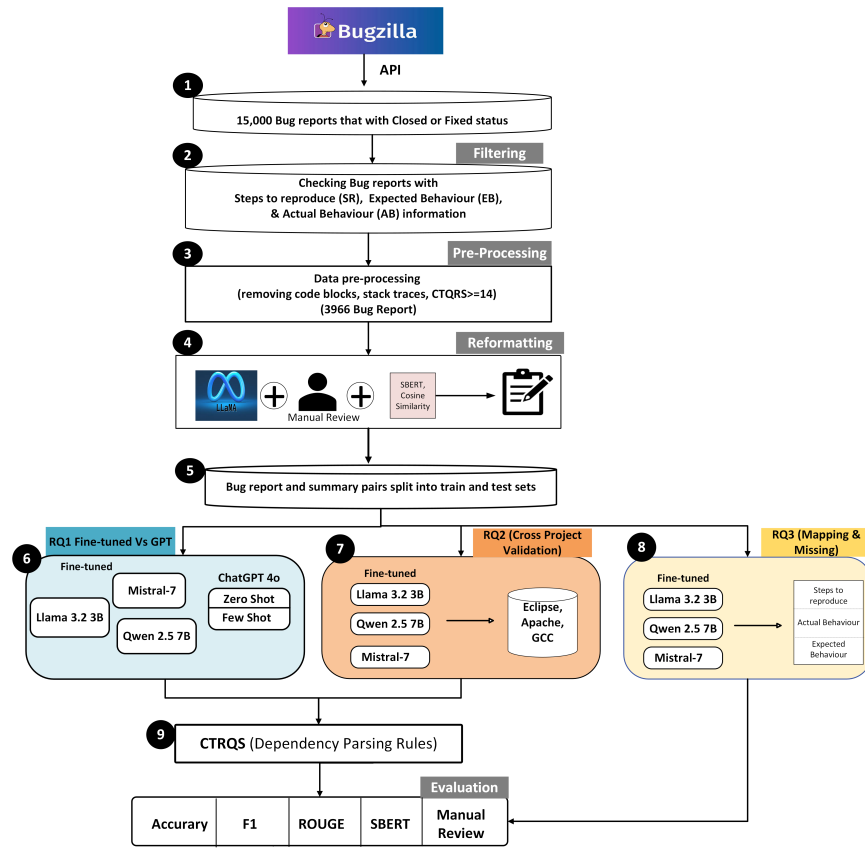


Figure 3: Architecture for Generating High-Quality Bug Reports from Unstructured Bug Reports Using Fine-Tuned Large Language Models

Listing 1: Llama 3 prompt example for generating unstructured bug report

Please rewrite the following bug report in a natural, conversational tone, as if you're explaining it to someone casually. Keep the essence of the report intact, but restructure it in a way that sounds like something an average person would write, while still using the original wording from the report as much as possible. Focus on maintaining the original details and key points without changing much. Provide only the one rewritten paragraph with everything, no additional explanation.

Bug report: {text}

experiments with different keywords and we got best results when we requested with please keyword and final evaluations to design the prompt as shown in Listing 1 (to generate an unstructured from

a well-structured bug report). Further, utilizing this prompt with the state-of-the-art Llama3 model [18], we generated summaries for all 3,966 well-structured bug reports in our training set. To ensure the generated reports were closely aligned with the original reports, we manually verified 200 reports computed SBERT [57] and cosine similarity [41] scores as shown in step 4 of Figure 3. Each unstructured report was generated three times, and only those with an SBERT similarity exceeding 85% and a cosine similarity above 80% were retained. The final dataset comprised 3,903 well-structured bug reports paired with their summaries, serving as synthetic pseudo-ground truth for instruction fine-tuning tasks.

Data Splitting: Step 5 in Figure 3 illustrates the data splitting process, where the data was randomized and then split into training, testing and validation sets, where training was 80% of the data, comprising 3,122 rows, testing 10% with 391 rows, and the remaining 10% was validation with 390 rows. We finetuned our model using 4 cross-validation.

Prompt Design: A prompt [44] serves as a set of instructions that directs LLMs to generate a specific desired output [4]. The effectiveness of an LLM's performance on the same task can vary depending on the prompt used [39], making it essential to craft

Listing 2: Alpaca prompt template used to fine-tune open-source LLMs.

```

alpaca_prompt You are a senior software
               engineer specialized in generating
               detailed bug reports.
### Instruction:
Please create a bug report that includes the
following sections:
1. Steps to Reproduce (S2R): Detailed steps
   to replicate the issue.
2. Expected Result (ER): What you expected
   to happen.
3. Actual Result (AR): What actually
   happened.
4. Additional Information: Include relevant
   details such as software version, build
   number, environment, etc.

If any of these sections are missing from
the provided report, explicitly notify
the user which information is missing.

### Input:
{unstructured_report}

### Response:
{Bug_report}

```

precise prompts. In our approach, we employ a single-round dialogue interaction to formulate prompts, utilizing the standard Alpaca-LoRA template [65] as shown in Listing 2. Additionally, we adopt a strategy similar to that used by Bo et al. [6] to create an effective prompt template for fine-tuning: (1) providing important task-related context as much as possible; (2) assigning LLMs a specified role for our task (Senior Software Engineer); (3) using separators in the prompt to indicate different parts of the input; (4) formatting the LLM’s output in a standardized JSON structure for better analysis; and (5) ensuring the prompt is both concise and accurate to fit within the LLM’s input token limitations. The prompts have been structured using the standard Alpaca-LoRA template [65] meticulously encoded through the model’s tokenizer. This includes adding the `<|begin_of_text|>` token (equivalent to the BOS token) and the `<|eot_id|>` token (which signifies the end of the message in turn). All the parameters used for this step are reported in the example of fine-tuning using Unsloth and the TRL SFTTrainer [20] available on our GitHub repository.²

Instruction fine-tuning: We used Parameter Efficient Fine-Tuning (PEFT), which doesn’t fine-tune the entire model but modifies several parameters to adapt the models for different applications [46]. This approach helps reduce the substantial expenses linked to full-fine-tuning and ensures that fine-tuning is feasible even with constrained storage and processing power. Low-rank adaptation (LoRA)

[31], a prominent PEFT technique, reduces trainable parameters by incorporating low-rank trainable matrices within the attention layers of the Transformer model and freezing the model’s weights.

We have fine-tuned the top three widely used models in the literature and Huggingface leaderboard: Qwen 2.5-7B, Mistral-7B, and Llama 3.2B [5, 7, 28, 32]. To improve efficiency and reduce resource consumption, we adopt the Unsloth framework to optimize the Low-Rank Adaptation (LoRA) method for fine-tuning the models, setting the rank to 16. We specifically target LoRA modules such as ‘q_proj’, ‘k_proj’, ‘o_proj’, ‘v_proj’, ‘down_proj’, ‘gate_proj’, and ‘up_proj’.

Following standard fine-tuning hyperparameters, we train the Mistral-7B and Qwen-2.5 B model for 3 epochs with a learning rate of $2e-4$ and a batch size of 8 examples. We also fine-tuned the Llama-3.2 3B models in the subsequent analysis. For these models, we conducted training for 3 epochs with a learning rate of $3e-3$ and a batch size of 8 samples.

The number of learning rate, LORA rank and epochs were determined based on experimental observations; we noticed that the model’s performance on the validation set plateaued after 3 epochs, indicating that additional training did not yield significant improvements and a 4-Cross validation was applied.

Implementation Details The hardware configuration used for fine-tuning was RTX 4090 GPU, with 32 GB of RAM. The models were fine-tuned using the Unsloth framework [66].

Evaluation metrics: As illustrated in 9, all models were evaluated using established metrics widely employed in similar studies [11, 43, 54, 76]: specifically, ROUGE-1 [42], SBERT [57] and Cosine Similarity [41], along with Accuracy and F1 Score.

Although ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is a widely used metric for summarization tasks, it is applicable to evaluating paraphrases. ROUGE-1 measures recall through matched unigrams, and we employ this variant in our assessments.

We implemented the bug report quality metrics score proposed by Zhang et al. [75], which evaluates reports based on Atomicity, Conciseness, Completeness, Understandability, and Reproducibility using dependency parsing.

For RQ1 6, we compared the quality of bug reports generated by fine-tuned models with those from ChatGPT. For RQ2 7, we assessed the generalization capability of the fine-tuned models. For RQ3 8, we evaluated the accuracy of the fine-tuned models for mappings of unstructured bug reports to high-quality bug reports and testing model’s missing information identification.

6 Results

We evaluated our fine-tuned models using CTQRS, ROUGE-1, and SBERT by passing unstructured test reports through the model to generate structured reports. The CTQRS score measures the quality of a bug report out of 17, based on the rules discussed in Section 3. As shown in Figure 4, the Qwen 2.5 model achieved an average score of approximately 77% on the test dataset, meaning the generated reports received an average of 13 out of 17 points. The ROUGE score is calculated by comparing unigrams from the generated reports with the actual ground truth to assess whether the model produces high-quality reports. SBERT is used to measure the

²https://github.com/GindeLab/Ease_2025_AI_model

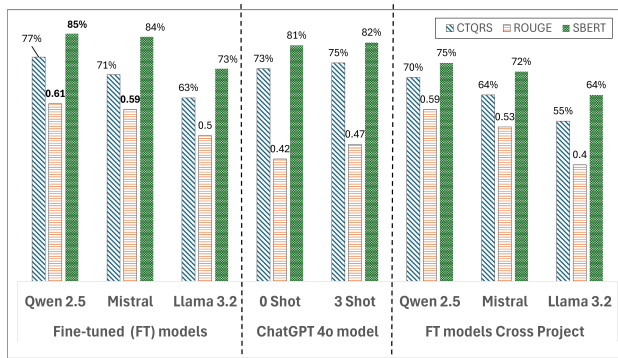


Figure 4: RQ1 and RQ2: Comparing the performance of fine-tuned models with base models and ChatGPT 4o on test dataset

semantic similarity between the model-generated reports and the ground truth, showing how closely the generated content matches the actual reports.

6.1 Answering RQ1: Fine-tuned models vs. ChatGPT 4o

As shown in the Figure 4 the fine-tuned Qwen2.5 model demonstrates superior performance across all metrics compared to other fine-tuned models. Specifically, the fine-tuned Qwen model achieved a CTQRS score of 77%, marking a significant improvement of 14% over its llama model's score of 63% and improvement of 5% as compared to ChatGPT. This enhancement trend is consistent across SBERT and ROUGE-1 scores, where the fine-tuned Qwen model outperforms all other fine-tuned models.

A key factor contributing to this improvement is Qwen's implementation of the Grouped-Query Attention (GQA) mechanism. This advanced attention mechanism provides a notable improvement over the standard attention mechanisms employed by similarly sized models, such as Mistral. Additionally, the fine-tuned Qwen model demonstrates a remarkable capability in generating high-quality bug reports from unstructured reports, surpassing lower-parameter models, performing close to SOTA model ChatGPT in post-fine-tuning performance [56].

RQ1: Comparing Fine Tune and ChatGPT shows fine-tuned models Qwen and Mistral models comparable to ChatGPT, achieving CTQRS scores of 77% and 71%, respectively, compared to 75% for ChatGPT. Additionally, both fine-tuned models surpass ChatGPT in ROUGE Score, with Qwen and Mistral attaining scores of 0.64 and 0.62, respectively, against 0.44 for ChatGPT.

6.2 Answering RQ2: Generalizability of Fine-tuned models

We manually curated a dataset of 300 high-quality reports from the publicly available dataset shared by Song et al. [60]. These reports were processed through our pipeline, following the methodology outlined in Section 5, and evaluated using the same approach as RQ1.

Missing info	Accuracy			F1		
	S2R	AB	EB	S2R	AB	EB
Models						
Qwen 2.5	74%	45%	47%	76%	45%	44%
Mistral	72%	49%	48%	73%	47%	46%
Llama 3.2	68%	52%	51%	69%	53%	52%
Mapping	ROUGE-1			METEOR		
Qwen 2.5	0.52	0.72	0.7	0.49	0.69	0.68
Mistral	0.45	0.7	0.69	0.41	0.66	0.65
Llama 3.2	0.39	0.56	0.55	0.37	0.55	0.53

Lower (Bad) Medium Higher (Good)

Figure 5: RQ3 – Heat-map. Upper part (“Missing info”): shows how accurately the model can flag missing fields (higher = better). Bottom part (“Mapping”): shows how well the model maps content from user text to structured report fields (higher = better).

Our results, presented in Figure 4, indicate that fine-tuned Qwen performed comparably to the ChatGPT model achieving 70% CTQRS score while ChatGPT achieved 73%. These findings highlight the effectiveness of fine-tuning for specific tasks, demonstrating that task-specific fine-tuned models can achieve similar performance to state-of-the-art (SOTA) models at a lower compute cost.

Additionally, we observed that ChatGPT's verbose text generation negatively impacted the overall evaluation score. Furthermore, three-shot prompting with ChatGPT outperformed zero-shot prompting, suggesting that providing more examples in the prompt improves evaluation scores. Further supporting this trend, Pham et al. [55] emphasized that while ChatGPT can be expensive to deploy for specific natural language generation tasks, fine-tuning smaller models on high-quality, in-domain datasets can lead to superior performance.

RQ2: Comparing fine-tuned models on other OSS projects, reveals that the fine-tuned Qwen model achieved a robust 70% CTQRS score, followed by Mistral with 64%. Significantly, this outperforms Llama3.2's score of 55% in CTQRS. These outcome emphasizes that the performance benefits observed with fine-tuned models are not limited to a single dataset. Instead, it indicates a valuable degree of generalizability, suggesting that fine-tuning provides a broadly effective strategy for enhancing model performance across diverse datasets.

6.3 Answering RQ3: Mapping and Missing information Detection

To determine the missing information score, we systematically masked different sections of the unstructured reports in the test dataset, including Steps to Reproduce, Actual Behavior, and Expected Behavior. This approach allowed us to evaluate whether the model could accurately identify if the report miss any information.

As shown in Figure 5, the model struggled to detect Actual Behavior and Expected Behavior in approximately 45–50% of cases. Instead, it inferred the missing details based on the available context. However, the Steps to Reproduce section was correctly identified in over 70% of cases for the Qwen and Mistral models.

To evaluate the Mapping Score, we compared the JSON output of the model for each section against the corresponding section in the actual report. As illustrated in Figure 5, Actual Behavior and Expected Behavior and Actual Behavior were mapped more accurately, achieving a ROUGE score of 0.72 for the Qwen 2.5 model. This higher scores is likely due to the shorter length of these sections. In contrast, the Steps to Reproduce section, being more detailed and lengthy, had a lower ROUGE and METEOR score of 0.52 & 0.49.

RQ3: The fine-tuned Llama3.2 model demonstrated a slight improvement over other fine-tuned models in identifying missing information, particularly with respect to Actual Results and Expected Results. Our manual analysis revealed that the Qwen and Mistral models tend to generate missing information by inferring from the available context rather than explicitly flagging it as missing to the user. We saw model-frequently generated content for Actual Behavior opposite of the original information provided in Expected Behavior and vice-versa. However, it accurately highlighted when the 'Steps to Reproduce' section was missing for the majority of the samples.

The mapping of unstructured reports to structured formats was performed efficiently by the Qwen and Mistral models. However, we observed a decline in performance, with ROUGE scores dropping from 0.72 for mapping Actual Behavior to 0.52 for Steps to Reproduce. Manual analysis indicated that this decrease was related to the length and details in the "Steps to Reproduce" section. The models exhibited a tendency to introduce additional information in the Steps to reproduce section, which further contributed to the reduced ROUGE and METEOR scores.

7 Discussion

Our analysis demonstrates that utilizing small language models can achieve performance comparable to state-of-the-art (SOTA) models such as ChatGPT. In addition to their competitive performance, these open-source models offer several advantages, including reduced computational requirements, enhanced scalability, and improved data privacy by mitigating concerns related to proprietary data usage.

In our study, we have used on the unstructured reports generated (pseudo-ground truth data) using Llama3 model. Llama3 model because of its exceptional performance, open-source nature, transparency, and scalability, which allow us to access, modify, and understand the underlying model architecture and training processes, making it popular among researchers. Due to the lack or absence of a dataset large enough for instruction fine-tuning LLMs, Llama3 provides a valuable alternative for our study.

Our approach demonstrated that LLMs can adapt to various projects and repositories without extensive retraining. This makes our fine-tuned model a valuable tool for different open-source projects, enhancing collaboration and efficiency in software development. The ability of LLMs to understand and generate natural language makes them particularly suited for tasks like bug report generation, where clear and precise communication is essential.

Our LLM models, fine-tuned on the pseudo-ground truth dataset successfully generated bug reports in the required format for corresponding unstructured report. Additionally, the model successfully

identified and highlighted any missing information according to the bug report template. One such example is shown in Figure 6. This ability to generate and evaluate bug reports highlights the utility of LLMs in streamlining the bug-reporting process. The novelty of our approach lies in leveraging LLMs to create bug reports that adhere to bug report templates automatically and ensure they meet specific formatting and informational standards.

8 Threats to validity

In this section, we list various limitations of our study and explain how we address them.

Internal threats: Our training dataset consists of 3,162 bug reports, which might raise concerns related to sample adequacy for analysis and model fine-tuning. However, Majdik et al. [45] demonstrated that a training set of around 2,500 samples can significantly enhance performance in domain-specific tasks like named entity recognition, summarization, and text generation.

Construct threats: The eports could introduce biases or inconsistencies in the model's outputs. If the prompts are not carefully crafted to align with the desired output format or guide the model to generate unexpected responses, this could skew the results. Large Language Models like Llama 3 can sometimes hallucinate or produce incorrect unstructured reports, which may impact the accuracy of bug reports and influence our results. Additionally, these models are highly dependent on the prompts given to them; slight changes in the input can lead to significantly different outputs. This sensitivity introduces variability and potential bias into the data generation process, affecting the reliability of our conclusions. We mitigate this threat by carefully designing our prompts using the Alpaca-LoRA template [65] and keeping them consistent throughout the fine-tuning process.

Furthermore, LLMs are prone to hallucination, generating plausible-sounding but factually incorrect or unsupported details—especially when inferring missing sections. In our manual review, we observed instances where the model filled in fields (e.g., actual vs. expected behavior) with contextually plausible yet inaccurate content. Future work will incorporate human validation steps and automatic consistency checks to detect and filter out hallucinated information. To mitigate the risk of data leakage, we mined the dataset of recent bug reports from bugzilla from November 2024, ensuring that the LLMs had not been exposed to similar data during their pre-training. While computing evaluation measures, external variables such as differences in the length of the bug report generated can lead to unfair comparisons in n-gram overlap metrics, as models producing lengthier or detailed reports may provide helpful information that does not align with well-structured bug reports, thereby lowering their scores. We tried to mitigate this issue by introducing additional grading metrics i.e SBERT and CTQRS. Furthermore, the quality of data preparation and reference unstructured reports (pseudo-ground truth data) has an impact on the overall results.

External threats: The models, fine-tuned specifically for bug reports belonging to the Mozilla family of projects and bug report guidelines, thus, may not perform as effectively or as relevant in other organizational contexts or with different types of reports, limiting their broader applicability until they are fine-tuned per

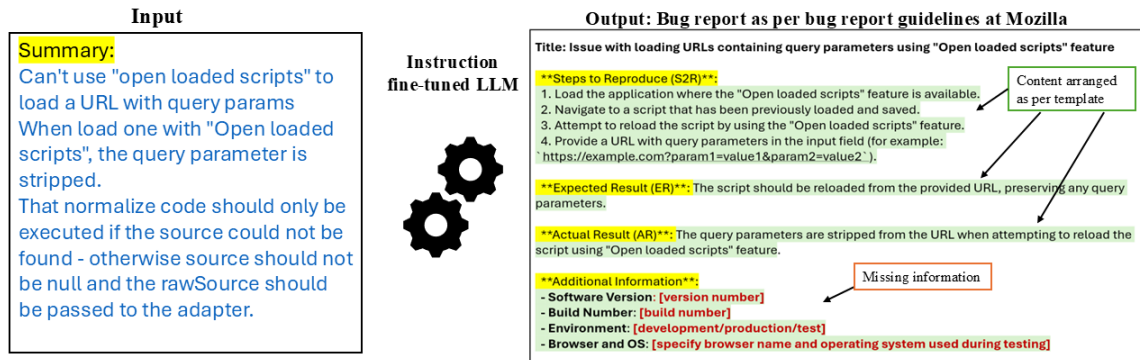


Figure 6: Sample output from our proposed solution approach: Content gets automatically organized as per the template, and missing information is also displayed for further editing for bug reporter

the organization's guidelines. However, generic testing on samples sourced from [51] showed encouraging results, thus mitigating this threat to a large extent.

Construct threats: The evaluation metrics ROUGE-1, CTQRS and METEOR emphasize structural or lexical overlap and can underestimate semantic quality for example, penalizing a perfectly clear paraphrase that uses different wording. To address this, we plan to integrate human-judged evaluations and explore specialized metrics that better capture developer-perceived clarity, usefulness, and actionability of bug reports in future work. Incorporating human evaluations or developing specialized metrics tailored to bug report generation could offer a more accurate assessment, which will be part of our future work.

9 Related work

We discuss related work with respect bug report quality and instruction fine-tuning of LLMs as follows.

Bug Report Management (BRM): Bug report management has evolved through studies focusing on quality assessment and automation of software engineering tasks. Bettenburg et al. [77] were the first who explore this issue and analyzed the summary, steps to duplicate, and test cases in the bug report using traditional ML techniques to focus on the quality aspects of bug reports. Zanetti et al. [74] further checked for duplicate images to ensure the quality of the description text and developed methods to assess report validity. Chaparro et al. [12] explored quality assessment of S2Rs using grammar parsing, neural sequence labelling, and matching of S2Rs to graph-based execution models of the Android application

Since the advancement in NLP, bug report Summarization has been explored widely. Kou et al. [38] used three sentence significance factors, i.e. believability, sentence-to-sentence cohesion, and topic association, to summarize the bug report. Later, Xiang and Shao [71] proposed SumLLaMA, which used LLMs to generate summaries from bug reports.

The significant manual effort required for bug triage and resolution [1, 25, 77]. Breu et al. [8] explored the information requirements in bug reports by studying the questions posed in 600 bug reports from the Mozilla and Eclipse projects. Additionally, incomplete

or unclear S2Rs pose a big challenge for automated methods to generate test cases from bug reports [23, 35]. While multiple NLP approaches have been explored, as discussed above, the use of instruction fine-tuning on large language models (LLMs) to improve bug report quality has not yet been investigated. This represents a significant research gap, which our study aims to address.

Bug report quality: It has been widely studied, with various methods proposed to evaluate and improve it [27, 77]. Many of these methods use heuristic rules and expert insights to identify key details in bug reports. He et al. [29] introduced a convolutional neural network (CNN)-based approach to classify bug reports as valid or invalid using only textual data, such as summaries and descriptions. Similarly, Chen et al. [16] leveraged natural language processing and quantifiable indicators to assess bug report quality automatically. Building on this, we adopted the **CTQRS** framework by Zhang et al. [75], which is a bug-report quality assessment framework that systematically scores bug reports by combining morphological, relational, and analytical indicators through dependency parsing. Morphological indicators (size, readability, punctuation) capture structural and linguistic aspects; relational indicators (itemization, complete environment info, screenshots) examine whether each standard field is properly provided; and analytical indicators (interface elements, user behavior, system defects) tap into deeper semantics by checking how clearly a report describes UI elements, actions, and defect details. Each indicator is linked to desirable properties of atomicity, completeness, conciseness, understandability, and reproducibility and is assigned rule-based weights to compute an overall quality score with a maximum score of 17 points.

Instruction fine-tuning: Large pre-trained language models are capable of executing a diverse variety of generative tasks utilizing human-annotated instruction data. Notable examples of such tasks include story writing [73], email drafting, the design of menu systems [36], poetry composition [10], code generation [50], and the creation of food recipes [26]. Taking inspiration from these studies, we explore instruction fine-tuning.

10 Conclusion and Future Work

In this paper, we demonstrated how instruction fine-tuned LLMs can automatically convert casual, unstructured bug reports into well-structured ones that closely follow standard templates. Our experiments revealed that Qwen 2.5, when fine-tuned, consistently outperformed other open-source models (Mistral 7B and Llama 3.2) across multiple evaluation metrics, achieving a CTQRS score of 77%, ROUGE-1 score of 0.64, and SBERT similarity of 0.82. These results underscore its ability to generate structured reports that closely align with human-written outputs, matching the performance of proprietary models like ChatGPT (75% CTQRS in 3-shot learning).

A key contribution of this work lies in addressing the challenge of inconsistent bug report quality, which hinders software maintenance efficiency. By leveraging instruction fine-tuning, our approach ensures that critical components such as Steps to Reproduce, Expected & Actual Behavior are systematically captured, even when unstructured input lack explicit details. This capability reduces developer effort by clarifying ambiguous reports and helps reporters identify and supply missing information.

Furthermore, our cross-project evaluation highlighted the robust generalization of model's. When tested on bug reports from diverse ecosystems (e.g., Eclipse, GCC, Apache), Qwen 2.5 maintained a CTQRS score of 70%, proving its adaptability to varying project contexts. Thus showcasing the efficacy of instruction fine-tuning on open-source models, we provide evidence that such models can serve as cost-effective, privacy-preserving, scalable alternatives to proprietary systems like ChatGPT, without compromising much on performance.

In the future, we will enhance the capabilities of model's by integrating richer data sources (e.g., error snapshots, logs, and code snippets), exploring advanced fine-tuning techniques such as QLoRA, expanding support for additional bug-tracking platforms (e.g. Jira and GitHub), and developing a real-time tool that proactively assists reporters in providing missing details. These improvements aim to enhance the bug-reporting experience and minimize developers' manual effort.

Acknowledgments

This research is supported by NSERC Discovery grant RGPIN-2023-03365. Also, we thank the anonymous reviewers for their constructive feedback which has helped improve this work further.

References

- [1] Jagrit Acharya and Gouri Ginde. 2024. Graph Neural Network vs. Large Language Model: A Comparative Analysis for Bug Report Priority and Severity Prediction. In *Proceedings of the 20th International Conference on Predictive Models and Data Analytics in Software Engineering* (Porto de Galinhas, Brazil) (PROMISE 2024). Association for Computing Machinery, New York, NY, USA, 2–11. doi:10.1145/3663533.3664042
- [2] Iftekhar Ahmed, Nitin Mohan, and Carlos Jensen. 2014. The impact of automatic crash reports on bug triaging and development in mozilla. In *Proceedings of The International Symposium on Open Collaboration*. 1–8.
- [3] Jorge Aranda and Gina Venolia. 2009. The secret life of bugs: Going past the errors and omissions in software repositories. In *2009 IEEE 31st international conference on software engineering*. IEEE, 298–308.
- [4] Yejin Bang et al. 2023. A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity. *arXiv preprint arXiv:2302.04023* (2023).
- [5] Edward Beeching et al. 2023. Open LLM Leaderboard. https://huggingface.co/spaces/open-llmleaderboard/open_llm_leaderboard. (Accessed: [Insert Date of Access]).
- [6] Lili Bo et al. 2024. ChatBR: Automated Assessment and Improvement of Bug Report Quality Using ChatGPT. In *2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1472–1483.
- [7] Z. Bouhoun et al. 2024. CurieLM: enhancing large language models for nuclear domain applications. *EPJ Web of Conferences* 302 (2024), 17006. doi:10.1051/epjconf/202430217006
- [8] Silvia Breu, Rahul Premraj, Jonathan Sillito, and Thomas Zimmermann. 2010. Information needs in bug reports: improving cooperation between developers and users. In *Proceedings of the 2010 ACM conference on Computer supported cooperative work*. 301–310.
- [9] Zeju Cai, Jianguo Chen, et al. 2024. F-CodeLLM: A Federated Learning Framework for Adapting Large Language Models to Practical Software Development. *2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)* (2024), 416–417.
- [10] Tuhin Chakrabarty, Vishakh Padmakumar, and He He. 2022. Help me write a poem: Instruction tuning as a vehicle for collaborative poetry writing. *arXiv preprint arXiv:2210.13669* (2022).
- [11] Chieh-Ju Chao, Imon Banerjee, et al. 2024. Evaluating Large Language Models in Echocardiography Reporting: Opportunities and Challenges. *medRxiv* (2024). doi:10.1101/2024.01.18.24301503
- [12] Oscar Chaparro et al. 2019. Assessing the quality of the steps to reproduce in bug reports. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Tallinn, Estonia) (ESEC/FSE 2019)*. Association for Computing Machinery, New York, NY, USA, 86–96. doi:10.1145/3338906.3338947
- [13] Oscar Chaparro, Carlos Bernal-Cárdenas, Jing Lu, Kevin Moran, Andrian Marcus, Massimiliano Di Penta, Denys Poshyvanyk, and Vincent Ng. 2019. Assessing the quality of the steps to reproduce in bug reports. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Tallinn, Estonia) (ESEC/FSE 2019)*. Association for Computing Machinery, New York, NY, USA, 86–96. doi:10.1145/3338906.3338947
- [14] Oscar Chaparro, Jing Lu, Fiorella Zampetti, Laura Moreno, Massimiliano Di Penta, Andrian Marcus, Gabriele Bavota, and Vincent Ng. 2017. Detecting missing information in bug descriptions. In *Proceedings of the 2017 11th joint meeting on foundations of software engineering*. 396–407.
- [15] Ning Chen, Jialiu Lin, Steven CH Hoi, Xiaokui Xiao, and Boshen Zhang. 2014. AR-miner: mining informative reviews for developers from mobile app marketplace. In *Proceedings of the 36th international conference on software engineering*. 767–778.
- [16] Xin Chen, He Jiang, Xiaochen Li, Tiek He, and Zhenyu Chen. 2018. Automated quality assessment for crowdsourced test reports of mobile applications. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 368–379.
- [17] Hyung Won Chung et al. 2024. Scaling Instruction-Finetuned Language Models. *Journal of Machine Learning Research* 25, 70 (2024), 1–53. <http://jmlr.org/papers/v25/23-0870.html>
- [18] Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 Herd of Models. *arXiv:2407.21783 [cs.AI]* <https://arxiv.org/abs/2407.21783>
- [19] Mona Erfani Joorabchi, Mehdi Mirzaaghaei, and Ali Mesbah. 2014. Works for me! characterizing non-reproducible bug reports. In *Proceedings of the 11th working conference on mining software repositories*. 62–71.
- [20] Hugging Face. 2023. Supervised Fine-tuning Trainer. https://huggingface.co/docs/trl/sft_trainer. [Accessed 10-11-2024].
- [21] Hugging Face. 2023. unsloth/Llama-3.2-3B-Instruct. <https://huggingface.co/unsloth/Llama-3.2-3B-Instruct>. [Accessed 10-11-2024].
- [22] Hugging Face. 2023. unsloth/mistral-7b-instruct-v0.3. <https://huggingface.co/unsloth/mistral-7b-instruct-v0.3>. [Accessed 10-11-2024].
- [23] Mattia Fazzini, Martin Prammer, Marcelo d'Amorim, and Alessandro Orso. 2018. Automatically translating bug reports into test cases for mobile apps. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 141–152.
- [24] Zorik Gekhman et al. 2023. TrueTeacher: Learning Factual Consistency Evaluation with Large Language Models. In *Conference on Empirical Methods in Natural Language Processing*.
- [25] Philip J Guo, Thomas Zimmermann, Nachiappan Nagappan, and Brendan Murphy. 2010. Characterizing and predicting which bugs get fixed: an empirical study of microsoft windows. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering—Volume 1*. 495–504.
- [26] Helena H. Lee, Ke Shu, Palakorn Achananuparp, Philips Kokoh Prasetyo, Yue Liu, Ee-Peng Lim, and Lav R Varshney. 2020. RecipeGPT: Generative pre-training based cooking recipe generation and evaluation system. In *Companion Proceedings of the Web Conference 2020*. 181–184.
- [27] Rui Hao, Yang Feng, James A Jones, Yuying Li, and Zhenyu Chen. 2019. CTRAS: Crowdsourced test report aggregation and summarization. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 900–911.
- [28] Will Hawkins, Brent Mittelstadt, and Chris Russell. 2024. The effect of fine-tuning on language model toxicity. *arXiv:2410.15821 [cs.AI]* <https://arxiv.org/abs/2410.15821>

- 15821
- [29] Jianjun He, Ling Xu, Yuanrui Fan, Zhou Xu, Meng Yan, and Yan Lei. 2020. Deep learning based valid bug reports determination and explanation. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 184–194.
 - [30] Or Honovich et al. 2023. Unnatural Instructions: Tuning Language Models with (Almost) No Human Labor. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Toronto, Canada, 14409–14428. doi:10.18653/v1/2023.acl-long.806
 - [31] Edward J. Hu et al. 2021. LoRA: Low-Rank Adaptation of Large Language Models. CoRR abs/2106.09685 (2021). arXiv:2106.09685 <https://arxiv.org/abs/2106.09685>
 - [32] Ashvini Kumar Jindal, Pawan Kumar Rajpoot, and Ankur Parikh. 2024. Birbal: An efficient 7B instruct-model fine-tuned with curated datasets. arXiv:2403.02247 [cs.CL] <https://arxiv.org/abs/2403.02247>
 - [33] Mona Erfani Joorabchi, Ali Mesbah, and Philippe Kruchten. 2013. Real challenges in mobile app development. In *2013 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. IEEE, 15–24.
 - [34] Sascha Just, Rahul Premraj, and Thomas Zimmermann. 2008. Towards the next generation of bug tracking systems. In *2008 IEEE symposium on visual languages and human-centric computing*. IEEE, 82–85.
 - [35] Gün Karagöz and Hasan Sözer. 2017. Reproducing failures based on semiformal failure scenario descriptions. *Software Quality Journal* 25 (2017), 111–129.
 - [36] Amir Hossein Kargaran et al. 2023. MenuCraft: Interactive Menu System Design with Large Language Models. arXiv preprint arXiv:2303.04496 (2023).
 - [37] Amy J Ko and Parmit K Chilana. 2010. How power users help and hinder open bug reporting. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 1665–1674.
 - [38] Youngji Koh, Sungwon Kang, and Seonah Lee. 2022. Deep Learning-Based Bug Report Summarization Using Sentence Significance Factors. *Applied Sciences* 12, 12 (2022). doi:10.3390/app12125854
 - [39] Takeshi Kojima et al. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.
 - [40] Abdullatif Köksal, Timo Schick, Anna Korhonen, and Hinrich Schütze. 2024. LongForm: Effective Instruction Tuning with Reverse Instructions. arXiv:2304.08460 [cs.CL] <https://arxiv.org/abs/2304.08460>
 - [41] Baoli Li and Liping Han. 2013. Distance Weighted Cosine Similarity Measure for Text Classification. In *International Conference on Intelligent Data Engineering and Automated Learning (IDEAL) (Lecture Notes in Computer Science, Vol. 8206)*. Springer, 611–618. doi:10.1007/978-3-642-41278-3_74
 - [42] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*. Association for Computational Linguistics, Barcelona, Spain, 74–81. <https://aclanthology.org/W04-1013>
 - [43] Can Liu. 2024. CPMI-ChatGLM: Parameter-Efficient Fine-Tuning ChatGLM With Chinese Patent Medicine Instructions. *Scientific Reports* (2024). doi:10.1038/s41598-024-56874-w
 - [44] Pengfei Liu et al. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM computing surveys* 55, 9 (2023), 1–35.
 - [45] Z. P. Majdik et al. 2024. Sample size considerations for fine-tuning large language models for named entity recognition tasks: methodological study. *Jmir Ai* 3 (2024), e52095. doi:10.2196/52095
 - [46] Sourab Mangrulkar et al. 2022. PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. <https://github.com/huggingface/peft>. [Accessed 10-11-2024].
 - [47] Nuno Marques, Rodrigo Rocha Silva, and Jorge Bernardino. 2024. Using ChatGPT in Software Requirements Engineering: A Comprehensive Review. *Future Internet* 16, 6 (2024). doi:10.3390/fi16060180
 - [48] Kevin Moran, Mario Linares-Vásquez, Carlos Bernal-Cárdenas, and Denys Poshyvanyk. 2015. Auto-completing bug reports for android applications. In *Proceedings of the 2015 10th joint meeting on foundations of software engineering*. 673–686.
 - [49] Mozilla. [n. d.]. Bug Writing Guidelines. <https://bugzilla.mozilla.org/page.cgi?id=bug-writing.html>. [Accessed 10-11-2024].
 - [50] Niklas Muennighoff et al. 2023. Octopack: Instruction tuning code large language models. arXiv preprint arXiv:2308.07124 (2023).
 - [51] Samal Mukhtar, Seonah Lee, and Jueun Heo. 2024. A Multidocument Summarization Technique for Informative Bug Summaries. *IEEE Access* 12 (2024), 158908–158926. doi:10.1109/ACCESS.2024.3487443
 - [52] Nafiseh Niqueghbal, Amir Hossein Kargaran, and Abbas Heydarnoori. 2024. GIRT-Model: Automated Generation of Issue Report Templates. In *21st IEEE/ACM International Conference on Mining Software Repositories (MSR)*. IEEE/ACM, Lisbon, Portugal. <https://doi.org/10.1145/3643991.3644906>
 - [53] OpenAI. 2023. ChatGPT (Version 3.5) [Large language model]. <https://chat.openai.com>. Accessed: 2024-10-08.
 - [54] Jacob Parnell, Inigo Jauregi Unanue, and Massimo Piccardi. 2022. A Multi-Document Coverage Reward for RELAXed Multi-Document Summarization. *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL 2022)* (March 2022). doi:10.48550/arxiv.2203.02894
 - [55] Minh Quang Pham, Sathish Reddy Indurthi, Shamil Chollampatt, and Marco Turchi. 2023. Select, prompt, filter: Distilling large language models for summarizing conversations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 12257–12265.
 - [56] Qwen, ;, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, and Chengyuan Li. 2025. Qwen2.5 Technical Report. arXiv:2412.15115 [cs.CL] <https://arxiv.org/abs/2412.15115>
 - [57] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, Hong Kong, China, 3982–3992. doi:10.18653/v1/D19-1410
 - [58] Advait Sarkar, Andrew D Gordon, Carina Negreanu, Christian Poeltz, Sruti Srinivasa Ragavan, and Ben Zorn. 2022. What is it like to program with artificial intelligence? arXiv preprint arXiv:2208.06213 (2022).
 - [59] Philipp Schuegerl, Juergen Rilling, and Philippe Charland. 2008. Enriching SE ontologies with bug report quality. In *Proc. 4th International Workshop on Semantic Web Enabled Software Engineering (SWESE'08)*.
 - [60] Yang Song and Oscar Chaparro. 2020. Bee: A tool for structuring and analyzing bug reports. In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 1551–1555.
 - [61] Lin Tan, Chen Liu, Zhenmin Li, Xuanhui Wang, Yuanzhan Zhou, and Chengxiang Zhai. 2014. Bug characteristics in open source software. *Empirical Softw. Engg.* 19, 6 (Dec. 2014), 1665–1705. doi:10.1007/s10664-013-9258-8
 - [62] Ruixiang Tang, Xiaotian Han, Xiaojian Jiang, and Xia Hu. 2023. Does Synthetic Data Generation of LLMs Help Clinical Text Mining? arXiv abs/2303.04360 (2023). <https://api.semanticscholar.org/CorpusID:257405132>
 - [63] M. H. Tanzil, J. Y. Khan, and G. Uddin. 2024. Chatgpt incorrectness detection in software reviews. *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (2024), 1–12. doi:10.1145/3597503.3639194
 - [64] Qwen Team. 2024. Qwen2.5: A Party of Foundation Models. <https://qwenlm.github.io/blog/qwen2.5/>
 - [65] Tloen. 2023. Alpaca-lora/templates/. <https://github.com/tloen/alpaca-lora/>. [Accessed 10-11-2024].
 - [66] Unsloth. 2023. Unsloth GitHub. <https://github.com/unslothai>. [Accessed 10-11-2024].
 - [67] Dhaval Vyas et al. 2014. Bug reproduction: A collaborative practice within software maintenance activities. In *COOP 2014-Proceedings of the 11th International Conference on the Design of Cooperative Systems, 27-30 May 2014, Nice (France)*. Springer, 189–207.
 - [68] Q. Q. Wang, C. Parnin, and A. Orso. 2015. Evaluating the usefulness of ir-based fault localization techniques. *Proceedings of the 2015 International Symposium on Software Testing and Analysis* (2015). doi:10.1145/2771783.2771797
 - [69] Yizhong Wang et al. 2022. Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 5085–5109. doi:10.18653/v1/2022.emnlp-main.340
 - [70] Yizhong Wang et al. 2023. How Far Can Camels Go? Exploring the State of Instruction Tuning on Open Resources. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 74764–74786. https://proceedings.neurips.cc/paper_files/paper/2023/file/ec6413875e4ab08d7bc4d8e225263398-Paper-Datasets_and_Benchmarks.pdf
 - [71] Bangmeng Xiang and Yunna Shao. 2024. SumLLaMA: Efficient Contrastive Representations and Fine-Tuned Adapters for Bug Report Summarization. *IEEE Access* 12 (2024), 78562–78571. doi:10.1109/ACCESS.2024.3397326
 - [72] Yi Yao, Jun Wang, Yabai Hu, Lifeng Wang, Yi Zhou, Jack Chen, Xuming Gai, Zhenming Wang, and Wenjun Liu. 2024. BugBlitz-AI: An Intelligent QA Assistant. arXiv:2406.04356 [cs.SE] <https://arxiv.org/abs/2406.04356>
 - [73] Ann Yuan, Andy Coenen, Emily Reif, and Daphne Ippolito. 2022. Wordcraft: story writing with large language models. In *Proceedings of the 27th International Conference on Intelligent User Interfaces*. 841–852.
 - [74] Marcelo Serrano Zanetti, Ingo Scholtes, Claudio Juan Tessone, and Frank Schweitzer. 2013. Categorizing Bugs with Social Networks: A Case Study on Four Open Source Software Communities. arXiv:1302.6764
 - [75] Huan Zhang et al. 2022. Automated Quality Assessment for Crowdsourced Test Reports Based on Dependency Parsing. In *2022 9th International Conference on Dependable Systems and Their Applications (DSA)*. 34–41.
 - [76] Ming Zhao et al. 2021. On the Evaluation of Vision-and-Language Navigation Instructions. *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume* (April 2021), 1302–1316. doi:10.48550/arxiv.2101.10504
 - [77] Thomas Zimmermann et al. 2010. What Makes a Good Bug Report? *IEEE Transactions on Software Engineering* 36 (09 2010), 618–643. doi:10.1109/TSE.2010.63